

IMPLEMENTATION OF INHERITANCE TYPES

1.c1 extends c2 (Single Class Inheritance)

Problem Statement

Demonstrate single class inheritance where subclass C1 inherits fields and methods from superclass C2.

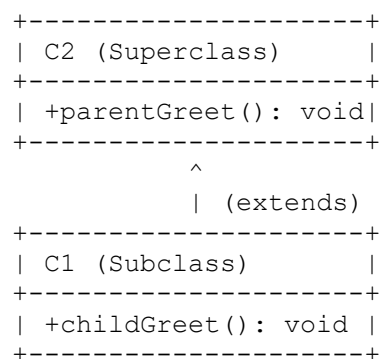
Steps

1. Define superclass C2 with a method `parentGreet()`.
2. Define subclass C1 extending C2 with `childGreet()`.
3. Instantiate C1 in `main()`.
4. Access both `parentGreet()` and `childGreet()` using the C1 object.

Algorithm

Step 1: Declare class C2 with `parentGreet()`.
Step 2: Declare class C1 extending C2 with `childGreet()`.
Step 3: Instantiate C1 in `main()`.
Step 4: Call `parentGreet()` via C1 instance (inherited).
Step 5: Call `childGreet()` via C1 instance (own method).

UML Class Diagram



Code

Java

// File: Scenario8.java

```
class C2 {
    void parentGreet() {
        System.out.println("Hello from Superclass C2!");
    }
}
```

```
// Single Inheritance: C1 inherits from C2
class C1 extends C2 {
    void childGreet() {
        System.out.println("Hello from Subclass C1!");
    }
}

public class Scenario8 {
    public static void main(String[] args) {
        C1 obj = new C1();

        // Inherited method call
        obj.parentGreet();

        // Own method call
        obj.childGreet();
    }
}
```

Output

```
Hello from Superclass C2!
Hello from Subclass C1!
```

2. c1 extends c2 and c3 (Multiple Class Inheritance)

Java does not support multiple inheritance with classes.

3. c1 implements i1 (Single Interface Implementation)

Problem Statement

Demonstrate single interface implementation where class c1 fulfills the abstract contract defined by interface i1.

Steps

1. Create an interface i1 with an abstract method display().
2. Create a class c1 that implements i1.
3. Provide a concrete implementation of display() in c1.
4. Instantiate c1 in main() and invoke display().

Algorithm

```
Step 1: Declare interface I1 with abstract method display().
Step 2: Define class C1 implementing I1.
Step 3: Implement display() inside C1 with public visibility.
Step 4: Create an object of C1 in main().
Step 5: Call display() using the object.
```

UML Class Diagram

Plaintext

```
+-----+
| <<interface>>      |
| I1                  |
+-----+
| +display(): void    |
+-----+
      ^
      | (implements)
+-----+
| C1                  |
+-----+
| +display(): void    |
+-----+
```

Code

Java

```
// File: Scenario2.java

interface I1 {
    void display();
}

class C1 implements I1 {
    @Override
    public void display() {
        System.out.println("C1 successfully implemented I1.");
    }
}

public class Scenario2 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.display();
    }
}
```

Output

```
C1 successfully implemented I1.
```

4.c1 implements i1, i2 (Multiple Interface Implementation)

Problem Statement

Demonstrate multiple interface implementation where class C1 satisfies contracts from two independent interfaces, I1 and I2.

Steps

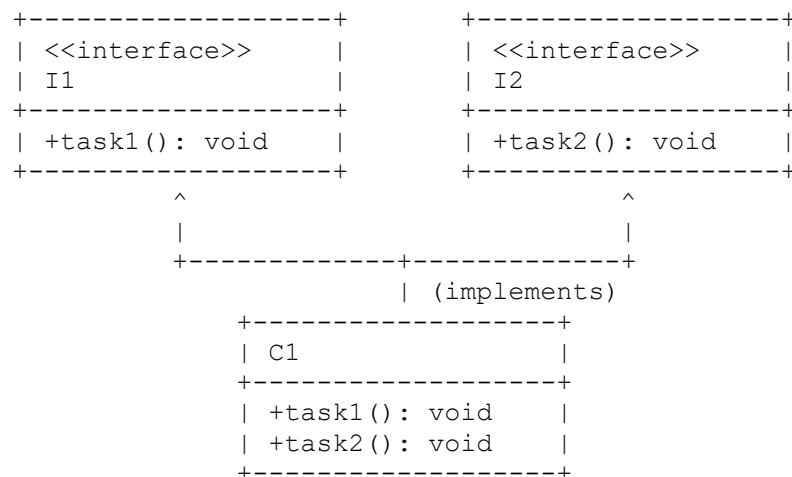
1. Define interface I1 with method `task1()`.
2. Define interface I2 with method `task2()`.
3. Define class C1 implementing both I1 and I2 separated by a comma.

4. Implement `task1()` and `task2()` inside `C1`.
5. Execute both methods from `main()`.

Algorithm

Step 1: Declare interface `I1` with `task1()`.
 Step 2: Declare interface `I2` with `task2()`.
 Step 3: Create class `C1` with clause: `implements I1, I2`.
 Step 4: Implement `task1()` and `task2()` in `C1`.
 Step 5: In `main()`, instantiate `C1` and call both tasks.

UML Class Diagram



Code

Java

```
// File: Scenario3.java

interface I1 {
    void task1();
}

interface I2 {
    void task2();
}

class C1 implements I1, I2 {
    @Override
    public void task1() {
        System.out.println("Task 1 from interface I1 executed.");
    }

    @Override
    public void task2() {
        System.out.println("Task 2 from interface I2 executed.");
    }
}

public class Scenario3 {
    public static void main(String[] args) {
        C1 obj = new C1();
    }
}
```

```

        obj.task1();
        obj.task2();
    }
}

```

Output

```

Task 1 from interface I1 executed.
Task 2 from interface I2 executed.

```

5.c1 implements c2 extends i1 (Invalid Keyword Syntax & Ordering)

Problem Statement

Demonstrate why `c1 implements c2 extends i1` fails in Java and fix the syntax by placing `extends` before `implements` and matching class/interface roles correctly.

6.i1 extends i2 (Single Interface Inheritance)

Problem Statement

Demonstrate single interface inheritance where sub-interface `I1` extends super-interface `I2`.

Steps

1. Define interface `I2` with `parentMethod()`.
2. Define interface `I1` that extends `I2` and adds `childMethod()`.
3. Create class `C1` implementing `I1`.
4. Implement both `parentMethod()` and `childMethod()` in `C1`.
5. Execute both methods from `main()`.

Algorithm

```

Step 1: Declare interface I2 with parentMethod().
Step 2: Declare interface I1 extending I2 with childMethod().
Step 3: Define class C1 implementing I1.
Step 4: Implement parentMethod() and childMethod() inside C1.
Step 5: In main(), instantiate C1 and call both methods.

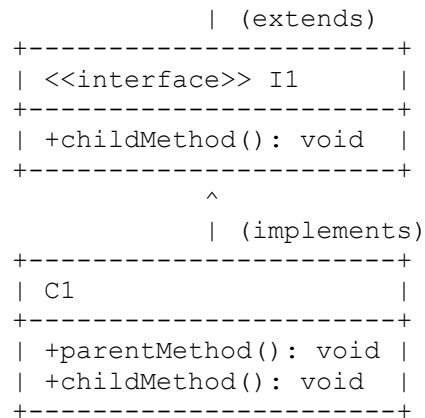
```

UML Class Diagram

```

+-----+
| <<interface>> I2      |
+-----+
| +parentMethod(): void |
+-----+
^

```



Code

Java

// File: Scenario5.java

```

interface I2 {
    void parentMethod();
}

// Interface extends Interface
interface I1 extends I2 {
    void childMethod();
}

class C1 implements I1 {
    @Override
    public void parentMethod() {
        System.out.println("Parent interface (I2) method implemented.");
    }

    @Override
    public void childMethod() {
        System.out.println("Child interface (I1) method implemented.");
    }
}

public class Scenario5 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.parentMethod();
        obj.childMethod();
    }
}

```

Output

```

Parent interface (I2) method implemented.
Child interface (I1) method implemented

```

7.i1 implements c1 (Invalid Interface Implementation)

Problem Statement

Demonstrate why an interface cannot implement a class in Java and show the correct reversed relationship (class C1 implements interface I1).

8.I1 extends I2 I3 (Multiple Interface Inheritance - Syntax Fix)

Problem Statement

Demonstrate multiple interface inheritance (I1 extending I2 and I3) and correct the syntax by adding a comma between extended interfaces.

Steps

1. Identify the missing comma between I2 and I3.
2. Write valid syntax: `interface I1 extends I2, I3`.
3. Implement I1 in concrete class C1.
4. Override all methods from I2, I3, and I1 inside C1.
5. Run the program from `main()`.

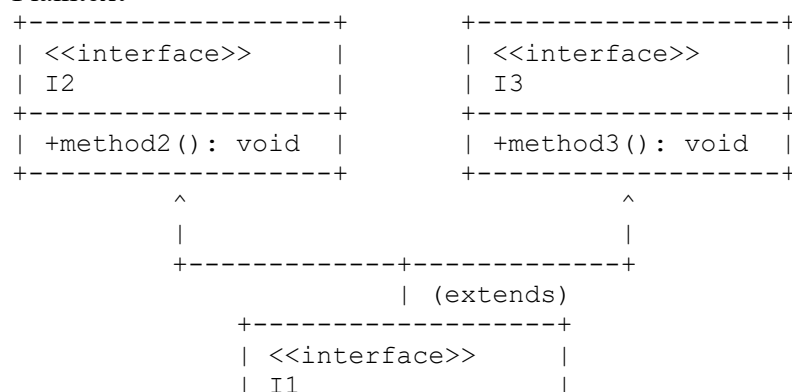
Algorithm

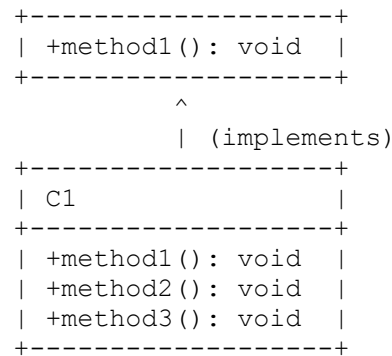
Plaintext

Step 1: Declare interface I2 with `method2()`.
Step 2: Declare interface I3 with `method3()`.
Step 3: Declare interface I1 extending I2, I3 with `method1()`.
Step 4: Create class C1 implementing I1.
Step 5: Provide implementations for `method1()`, `method2()`, and `method3()` in C1.
Step 6: In `main()`, instantiate C1 and call all three methods.

UML Class Diagram

Plaintext





Code

Java

```

// File: Scenario7.java

// INVALID SYNTAX: interface I1 extends I2 I3 (Missing comma)
// CORRECT SYNTAX: interface I1 extends I2, I3

interface I2 {
    void method2();
}

interface I3 {
    void method3();
}

// Interface extending multiple interfaces using comma separation
interface I1 extends I2, I3 {
    void method1();
}

class C1 implements I1 {
    @Override
    public void method2() {
        System.out.println("Method from I2 implemented.");
    }

    @Override
    public void method3() {
        System.out.println("Method from I3 implemented.");
    }

    @Override
    public void method1() {
        System.out.println("Method from I1 implemented.");
    }
}

public class Scenario7 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method2();
        obj.method3();
        obj.method1();
    }
}

```


Output

Plaintext

Method from I2 implemented.

Method from I3 implemented.

Method from I1 implemented.